National Institute of Technology Calicut
Department of Mechanical Engineering
ME3112: Metrology and Instrumentation

Uncertainty Analysis using the GUM Approach and Monte Carlo Simulation

Assignment report

Name: Chris Varghese Mathew **Roll No.:** B240029ME
Batch: ME-02 **Submitted:** 5 November 2026

Summary of results

Two measurement problems were evaluated in three independent ways: (i) the analytical GUM approach (law of propagation of uncertainty), worked by hand in section A; (ii) a Monte Carlo program written from scratch, with its own samplers for every distribution; and (iii) a Monte Carlo program produced by a generative AI tool (Claude (Anthropic) Opus 5.5). All Monte Carlo runs use $M = 2 \times 10^5$ trials.

Final results (95 % coverage)

	GUM (analytical)	Independent MCM code	AI MCM code
Problem 1 – Cylindricity C (μm)			
Estimate	8.3000	8.2991	8.3002
Std. uncertainty	0.3329	0.3404	0.3401
95 % interval	[7.6471, 8.9529]	[7.6335, 8.9669]	[7.6338, 8.9672]
Problem 2 – Zinc concentration c_{Zn} (mg/L)			
Estimate	2003.998	2003.986	2004.007
Std. uncertainty	2.586	2.586	2.589
95 % interval	[1998.929, 2009.066]	[1998.989, 2008.994]	[1999.027, 2009.012]

Problem 1: $C = (8.30 \pm 0.65)\,\mu$m, $u_c = 0.33\,\mu$m, $k = 1.96$ ($\nu_{\text{eff}} \approx 1965$), 95 % coverage.
Problem 2: $c_{Zn} = (2004.0 \pm 5.1)$ mg/L, $u_c = 2.6$ mg/L (0.13 % relative), $k = 1.96$, 95 % coverage.

All three methods agree on the estimates and on the standard uncertainty to within a few percent. The small systematic differences that remain are physical rather than numerical: in Problem 1 the Monte Carlo method gives a 2 % larger standard uncertainty because the Type A repeatability is represented by a t-distribution with only four degrees of freedom, and in Problem 2 the Monte Carlo interval is about 1.5 % narrower than the GUM interval because the output distribution is flatter than a Gaussian (it is dominated by a triangular and a rectangular input). These points are developed in section 6.

Source Code: https://github.com/pyrole23/uncertainty-analysis

Contents

Summary of results **1**

1 Introduction and objectives **4**

2 Theory **4**

2.1 Measurement model . . . 4

2.2 GUM analytical approach (JCGM 100) . . . 4

2.3 Monte Carlo method (JCGM 101) . . . 5

3 Problem 1: Cylindricity measurement **7**

3.1 Problem statement and measurement model . . . 7

3.2 GUM uncertainty . . . 7

3.3 Monte Carlo simulation . . . 7

4 Problem 2: Zinc concentration **10**

4.1 Problem statement and measurement model . . . 10

4.2 GUM uncertainty . . . 10

4.3 Monte Carlo simulation . . . 10

5 AI-generated program: prompts and verification **13**

5.1 Prompts used . . . 13

5.2 Verification of the generated code . . . 13

5.3 Corrections made . . . 14

6 Comparison and discussion **15**

6.1 Side-by-side comparison . . . 15

6.2 Agreement . . . 15

6.3 Differences and their causes . . . 15

6.4 Validation of GUM against MCM (JCGM 101, clause 8) . . . 16

6.5 Effect of interpretation choices . . . 17

7 Conclusions **17**

References **17**

A Analytical solution by the GUM method **18**

A.1 Problem 1: Cylindricity . . . 18

A.2 Problem 2: Zinc concentration . . . 20

B Raw program output **23**

B.1 Independent code: Problem 1 (`uncertainty_problem_1.py`) . . . 23

B.2 Independent code: Problem 2 (`uncertainty_problem_2.py`) . . . 23

B.3 AI-generated code (`uncertainty_analysis.py`) . . . 23

C Source code **25**

C.1 Independent code: Problem 1 (`uncertainty_problem_1.py`) 25

C.2 Independent code: Problem 2 (`uncertainty_problem_2.py`) 28

C.3 AI-generated code (`uncertainty_analysis.py`) 31

1 Introduction and objectives

Every measurement result is incomplete without a statement of its uncertainty. The *Guide to the Expression of Uncertainty in Measurement* (GUM, JCGM 100:2008) [3] gives an analytical framework based on a first-order Taylor expansion of the measurement model, while its Supplement 1 (JCGM 101:2008) [1] describes the *propagation of distributions* by a Monte Carlo method (MCM), which makes no linearity or normality assumptions [2]. This report applies both to the two problems in the assignment.

The tasks set in the assignment, and where each is addressed, are:

T1. Solve both problems with Monte Carlo simulation and with the analytical GUM approach, using a programming language (Python).

T2. Develop the GUM uncertainty budget: mean, standard uncertainties, sensitivity coefficients, combined standard uncertainty and expanded uncertainty at 95 %.

T3. Use a generative AI to write a second Monte Carlo program, and present the prompts used.

T4. Compare GUM, the independently written code and the AI-generated code; discuss agreement, differences and the reasons for discrepancies.

T5. Compute the 95 % interval with 2×10^5 trials and plot the histogram of the output PDF for each problem.

2 Theory

2.1 Measurement model

The measurand Y is related to the input quantities $X_1, \ldots, X_N$ through a measurement model

$$Y = f(X_1, X_2, \ldots, X_N). \tag{1}$$

Each input is described by a probability density function (PDF) that encodes the available knowledge about it: a best estimate x_i (its expectation) and a standard uncertainty $u(x_i)$ (its standard deviation). The two methods differ in what they propagate through f: GUM propagates only x_i and $u(x_i)$, MCM propagates the complete PDFs.

2.2 GUM analytical approach (JCGM 100)

Type A evaluation

For n independent repeated observations $q_1, \ldots, q_n$,

$$\bar{q} = \frac{1}{n}\sum_{j=1}^{n} q_j, \qquad s^2(q) = \frac{1}{n-1}\sum_{j=1}^{n}(q_j - \bar{q})^2, \qquad u(\bar{q}) = \frac{s(q)}{\sqrt{n}}, \qquad \nu = n - 1. \tag{2}$$

Type B evaluation

Type B uncertainties are derived from an assumed PDF $g(x)$. The standard uncertainty is the square root of the variance $u^2 = \int (x-\mu)^2 g(x)\,dx$, which for the PDFs in this assignment gives the closed forms in table 1 (derived in section A). In the independently written code these variances

are deliberately obtained by numerical integration of $\int x^2 g(x)\,dx$ (midpoint rule, $N = 10^4$ strips); the printed values agree with the closed forms to about 10^{-8} relative.

Table 1: Standard uncertainty of the input PDFs used in this work.

PDF	Parameters	Standard uncertainty	Used for
Normal (Gaussian)	expanded U, coverage factor k	$u = U/k$	calibration, mass, temperature
Rectangular (uniform)	half-width a, i.e. $[\mu - a, \mu + a]$	$u = a/\sqrt{3}$	resolution, purity, repeatability
Symmetric triangular	half-width a	$u = a/\sqrt{6}$	alignment, volume filling
Scaled-shifted t_ν (MCM)	$\bar{q}$, $s/\sqrt{n}$, $\nu = n - 1$	$u = \frac{s}{\sqrt{n}}\sqrt{\frac{\nu}{\nu-2}}$	repeatability (Problem 1)

Law of propagation of uncertainty

For uncorrelated inputs, a first-order Taylor expansion of f about the estimates gives

$$u_c^2(y) = \sum_{i=1}^{N} c_i^2\, u^2(x_i), \qquad c_i = \left.\frac{\partial f}{\partial X_i}\right|_{X=x}, \tag{3}$$

where c_i is the sensitivity coefficient and $|c_i|u(x_i)$ is the contribution of input i to the combined uncertainty. For a product/quotient model such as $c = 1000\,mP/V$ this becomes the convenient relative form

$$\left(\frac{u_c(c)}{c}\right)^2 = \left(\frac{u(m)}{m}\right)^2 + \left(\frac{u(P)}{P}\right)^2 + \left(\frac{u(V)}{V}\right)^2. \tag{4}$$

Effective degrees of freedom and expanded uncertainty

The Welch–Satterthwaite formula gives the effective degrees of freedom of u_c,

$$\nu_{\text{eff}} = \frac{u_c^4(y)}{\displaystyle\sum_{i=1}^{N} \frac{(c_i u(x_i))^4}{\nu_i}}, \qquad \nu_i = \infty \text{ for Type B inputs,} \tag{5}$$

and the expanded uncertainty at coverage probability $p = 95\,\%$ is

$$U = k\, u_c(y), \qquad k = t_{0.975}(\nu_{\text{eff}}), \tag{6}$$

so that $y \pm U$ is the 95 % coverage interval under the assumption that $(Y - y)/u_c$ follows a t-distribution (a Gaussian for $\nu_{\text{eff}} \to \infty$, $k = 1.960$). The conventional choice $k = 2$ is also reported.

2.3 Monte Carlo method (JCGM 101)

The MCM replaces the linearisation by direct numerical propagation of the PDFs:

1. choose the number of trials M (here 2×10^5);
2. draw M independent samples $x_{i,r}$, $r = 1, \ldots, M$, from the PDF of every input;

3. evaluate the model for each draw, $y_r = f(x_{1,r}, \ldots, x_{N,r})$;
4. sort the y_r to obtain a discrete representation of the distribution function of Y;
5. take the estimate and standard uncertainty as the sample mean and standard deviation, $\tilde{y} = \frac{1}{M}\sum y_r, \quad u^2(\tilde{y}) = \frac{1}{M-1}\sum (y_r - \tilde{y})^2$;
6. take the 95 % coverage interval from the sorted values and plot the histogram of the y_r as an approximation to the output PDF.

Coverage intervals

With $q = pM$ (rounded as in JCGM 101, 7.7.2), the *probabilistically symmetric* interval is $[y_{(r)}, y_{(r+q)}]$ with $r = (M - q)/2$, i.e. the 2.5 % and 97.5 % quantiles. The *shortest* interval is the one among $[y_{(r)}, y_{(r+q)}]$, $r = 1, \ldots, M - q$, with the smallest width. For a symmetric output PDF both coincide.

Validation of GUM by MCM

JCGM 101, clause 8, compares the end-points of the GUM interval $y \pm U$ with the MCM interval. If u_c is written with n_{dig} significant digits as $c \times 10^l$, the numerical tolerance is $\delta = \frac{1}{2} \times 10^l$; GUM is validated if

$$d_{\text{low}} = |y - U - y_{\text{low}}| \le \delta \quad \text{and} \quad d_{\text{high}} = |y + U - y_{\text{high}}| \le \delta. \tag{7}$$

Sensitivity coefficients are also obtained numerically, by forward (Problem 1) and central (Problem 2) finite differences of the model function, which avoids hand-differentiation errors. The AI-generated code instead uses NumPy's built-in samplers and analytical derivatives.

3 Problem 1: Cylindricity measurement

3.1 Problem statement and measurement model

The cylindricity error of a machined component is measured five times on a cylindricity tester, with repositioning between trials: 8.2, 8.5, 8.1, 8.4 and 8.3 μm. The measurement model is

$$C = C_{\text{ind}} + \delta_{\text{cal}} + \delta_{\text{res}} + \delta_{\text{align}}, \tag{8}$$

with the input information in table 2. All corrections have zero expectation, so $C = C_{\text{ind}}$.

Table 2: Problem 1: input quantities.

Input	Type	PDF	Information
C_{ind} (repeatability)	A	t_4 (MCM)	mean of 5 readings
δ_{cal} calibration	B	Normal	0 μm, $U_{\text{cal}} = 0.6\,\mu$m, $k = 2$
δ_{res} resolution	B	Uniform	resolution 0.1 μm, bounded by $\pm 0.05\,\mu$m
δ_{align} alignment	B	Triangular	centred at 0, limits $\pm 0.30\,\mu$m

3.2 GUM uncertainty

The complete hand calculation is given in section A.1. Its result is summarised in table 3; the programs reproduce every entry (section B).

Table 3: Problem 1: GUM uncertainty ($c_i = 1$ for all inputs; values in μm).

Source	Estimate	Type	PDF	$u(x_i)$	$\lvert c_i \rvert u(x_i)$	ν_i	% of u_c^2
Repeatability C_{ind}	8.30	A	t	0.07071	0.07071	4	4.51
Calibration δ_{cal}	0	B	Normal	0.30000	0.30000	∞	81.20
Resolution δ_{res}	0	B	Uniform	0.02887	0.02887	∞	0.75
Alignment δ_{align}	0	B	Triangular	0.12247	0.12247	∞	13.53
$C = 8.300\,\mu$m,	$u_c = 0.3329\,\mu$m,	$\nu_{\text{eff}} = 1965$,	$k = 1.961$				
$U_{95} = 0.653\,\mu$m,	**95 % interval** [7.647, 8.953] μm						

The calibration uncertainty dominates the budget (81 % of u_c^2), followed by the alignment (14 %); the five-reading repeatability contributes only 4.5 % and the resolution is negligible.

3.3 Monte Carlo simulation

Each input is sampled $M = 2 \times 10^5$ times: $C_{\text{ind}} = 8.30 + (s/\sqrt{5})\,t_4$, $\delta_{\text{cal}} \sim \mathcal{N}(0, 0.3^2)$, $\delta_{\text{res}} \sim \mathcal{U}(-0.05, 0.05)$ and $\delta_{\text{align}} \sim \text{Tri}(-0.3, 0, 0.3)$; the four arrays are added to obtain M values of C.

Table 4: Problem 1: Monte Carlo results ($M = 2 \times 10^5$).

Quantity	**Independent code** (seed 2026)	**AI-generated code** (seed 42)
Mean (μm)	8.29911	8.30025
Standard uncertainty (μm)	0.34040	0.34014
95 % symmetric interval (μm)	[7.63346, 8.96693]	[7.63375, 8.96721]
95 % shortest interval (μm)	–	[7.64015, 8.97317]
Half-width (μm)	0.66674	0.66673

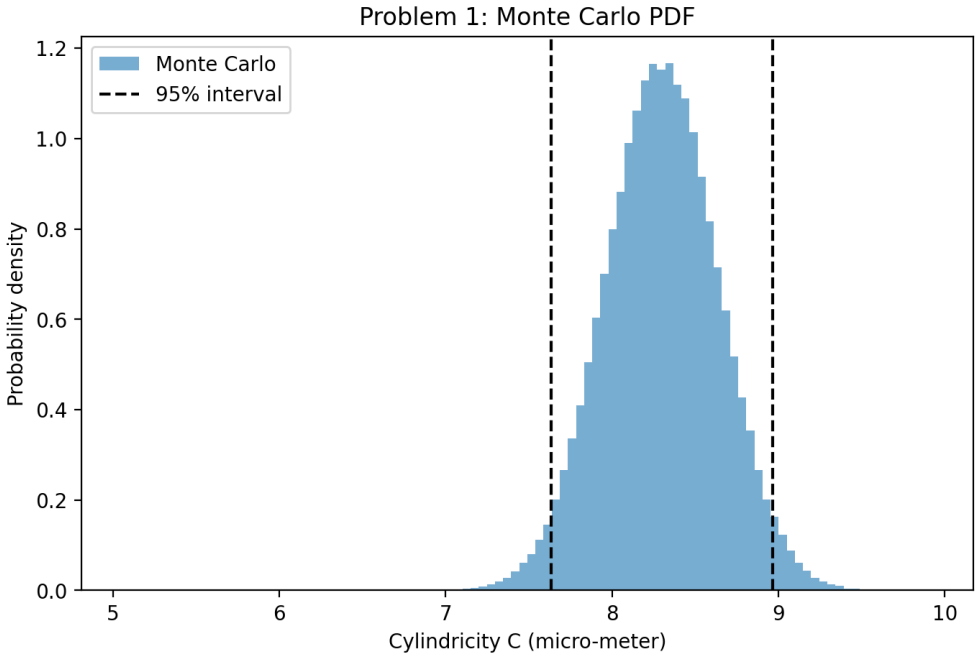


Figure 1: Problem 1: Histogram of the Monte Carlo PDF of cylindricity from the independently written code, with the 95 % coverage interval [7.633, 8.967] μm. The long, sparsely populated tails (out to about 5 and 10 μm) come from the heavy-tailed t_4 representation of the repeatability.

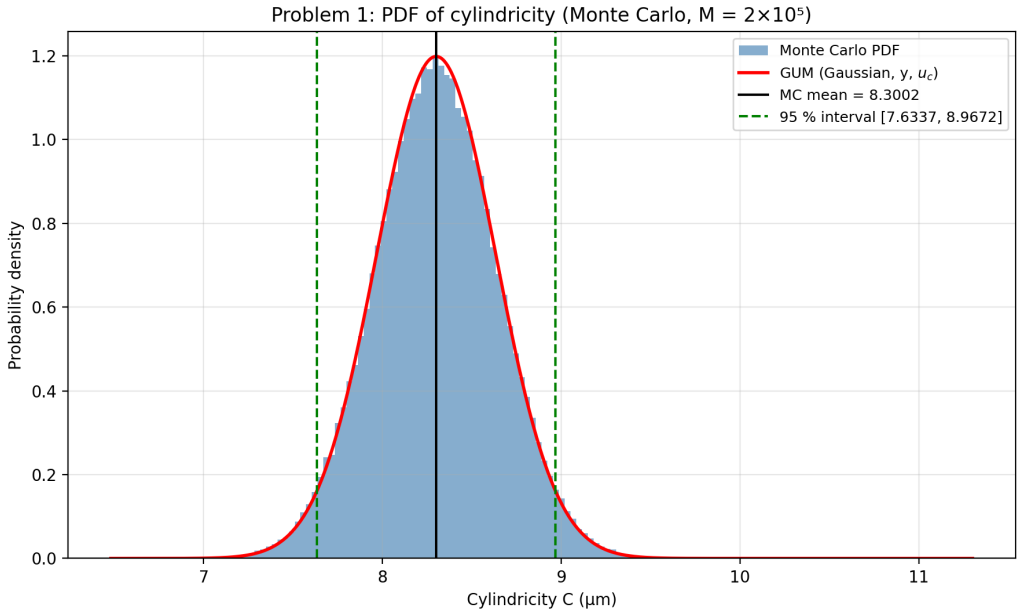


Figure 2: Problem 1: Monte Carlo PDF from the AI-generated code, with the GUM Gaussian $\mathcal{N}(8.30,\ 0.333^2)$ overlaid and the 95 % interval $[7.634,\ 8.967]\,\mu$m. The MCM histogram is slightly lower at the peak and wider than the GUM curve; the axis extends to about 11 μm because of a few extreme draws from the t_4 repeatability.

Problem 1 result. GUM: $C = (8.30 \pm 0.65)\,\mu$m ($k = 1.96$, $p = 95\,\%$). MCM: $C = 8.30\,\mu$m, $u = 0.34\,\mu$m, 95 % coverage interval $[7.63,\ 8.97]\,\mu$m.

4 Problem 2: Zinc concentration

4.1 Problem statement and measurement model

A high-purity zinc sample is weighed and dissolved in a solvent. The concentration is

$$c_{Zn} = \frac{1000\, m\, P}{V} \quad [\text{mg/L}], \tag{9}$$

with m in mg, P dimensionless and V in mL; the volume has three components, $V = V_{\text{fill}} + \delta V_{\text{rep}} + \delta V_{\text{temp}}$. The input information is in table 5.

Table 5: Problem 2: input quantities and their standard uncertainties.

Input	Type	PDF	Parameters	Estimate	$u(x_i)$
Purity P	B	Uniform	[0.9985, 1.0005]	0.9995	$0.001/\sqrt{3} = 5.774 \times 10^{-4}$
Mass m	B	Gaussian	mean 300.75, SD 0.009 mg	300.75 mg	0.009 mg
V: filling	B	Triangular	[149.6, 150.4], mode 150 mL	150 mL	$0.4/\sqrt{6} = 0.16330$ mL
V: repeatability	A	Rectangular	range 0.09 mL ($\pm$0.045)	0	$0.045/\sqrt{3} = 0.02598$ mL
V: temperature	B	Normal	0.1 mL at 95 %	0	$0.1/1.96 = 0.05102$ mL

The "range" of 0.09 mL for repeatability is interpreted as the full width of the rectangular distribution (half-width $a = 0.045$ mL), and the temperature uncertainty "0.1 mL at 95 %" is converted with $k = 1.96$ for a normal PDF. Both programs use the same interpretation; its effect is examined in section 6.5.

4.2 GUM uncertainty

The complete hand calculation is given in section A.2; the result is summarised in table 6.

Table 6: Problem 2: GUM uncertainty.

Source	Estimate	Type	PDF	$u(x_i)$	c_i	$\lvert c_i \rvert u(x_i)$ (mg/L)	% of u_c^2
Purity P	0.9995	B	Uniform	5.774×10^{-4}	2005.0	1.1576	20.03
Mass m	300.75 mg	B	Gaussian	0.009 mg	6.6633	0.0600	0.05
V filling	150 mL	B	Triangular	0.16330 mL	−13.360	2.1817	71.16
V repeatability	0	A	Rectangular	0.02598 mL	−13.360	0.3471	1.80
V temperature	0	B	Normal	0.05102 mL	−13.360	0.6816	6.95
$c_{Zn} = 2003.998$ **mg/L,** $u_c = 2.586$ **mg/L (0.129 %),** $\nu_{\text{eff}} = \infty$, $k = 1.96$							
$U_{95} = 5.069$ **mg/L,** **95 % interval** [1998.929, 2009.066] **mg/L**							

The volume-filling tolerance alone accounts for 71 % of the variance and the purity for 20 %; the mass contribution (0.05 %) is negligible.

4.3 Monte Carlo simulation

For each trial, $P \sim \mathcal{U}(0.9985, 1.0005)$, $m \sim \mathcal{N}(300.75, 0.009^2)$ and $V = \text{Tri}(149.6, 150, 150.4) + \mathcal{U}(-0.045, 0.045) + \mathcal{N}(0, 0.05102^2)$ are drawn and the nonlinear model is evaluated directly.

Table 7: Problem 2: Monte Carlo results ($M = 2 \times 10^5$).

Quantity	**Independent code** (seed 2026)	**AI-generated code** (seed 42)
Mean (mg/L)	2003.9863	2004.0066
Standard uncertainty (mg/L)	2.5861	2.5886
95 % symmetric interval (mg/L)	[1998.9891, 2008.9941]	[1999.0268, 2009.0122]
95 % shortest interval (mg/L)	–	[1999.0491, 2009.0308]
Half-width (mg/L)	5.0025	4.9927

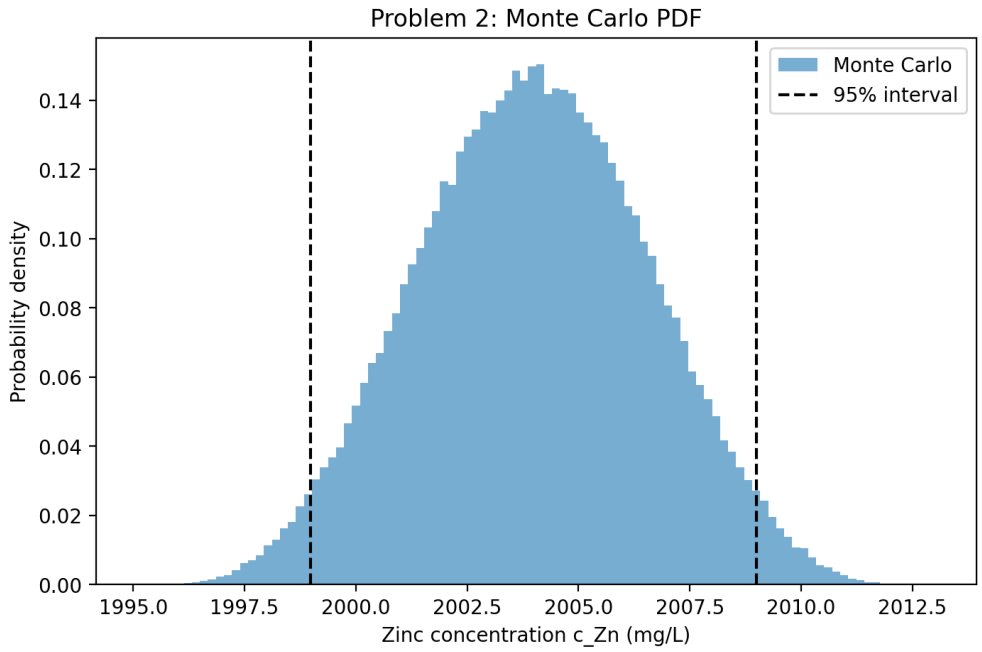


Figure 3: Problem 2: Histogram of the Monte Carlo PDF of zinc concentration from the independently written code, with the 95 % coverage interval [1998.99, 2008.99] mg/L.

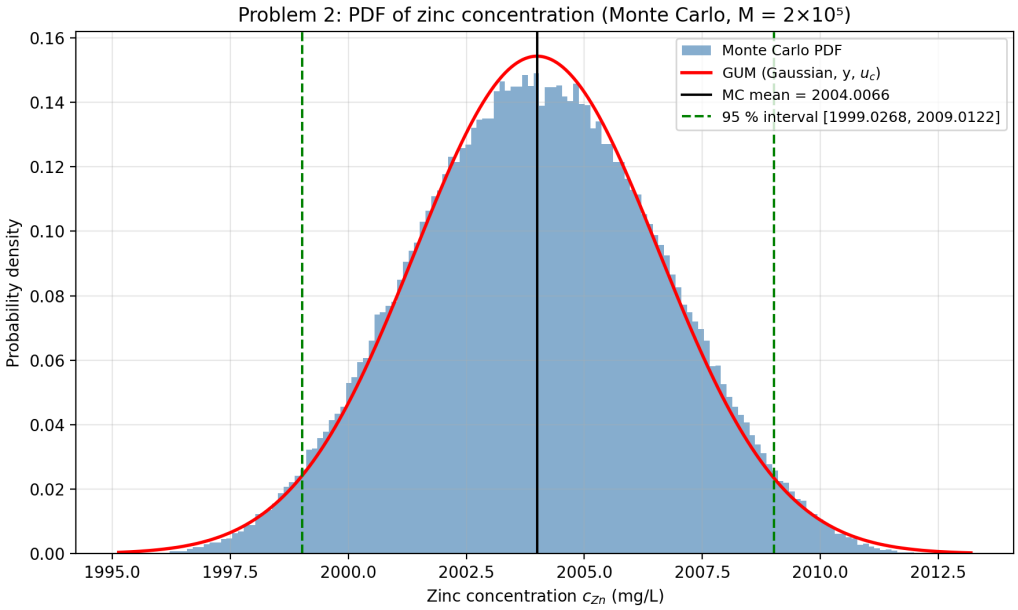


Figure 4: Problem 2: Monte Carlo PDF from the AI-generated code, with the 95 % interval [1999.03, 2009.01] mg/L and the GUM Gaussian $\mathcal{N}(2004.0,\ 2.586^2)$. The MCM histogram has a flatter top and shorter tails than the Gaussian, which is the signature of the dominant triangular and rectangular inputs.

Problem 2 result. GUM: $c_{Zn} = (2004.0 \pm 5.1)\,\text{mg/L}$ ($k = 1.96$, $p = 95\,\%$). MCM: $c_{Zn} = 2004.0\,\text{mg/L}$, $u = 2.6\,\text{mg/L}$, 95 % coverage interval $[1999.0,\ 2009.0]\,\text{mg/L}$.

5 AI-generated program: prompts and verification

5.1 Prompts used

The second Monte Carlo program (`uncertainty_analysis.py`) was generated with Claude (Anthropic) Opus 5.5. The prompts are reproduced below. The program solves both problems by the GUM method and by Monte Carlo simulation with NumPy's built-in random-number generators (seed 42); its results are compared with the hand calculation and the independently written code in section 6, and its raw output is given in section B.

Prompt 1 – main request

I am doing an assignment on uncertainty analysis for a Metrology and Instrumentation course. Write a single, well-commented Python program (NumPy, SciPy, Matplotlib) that solves the two problems below in two ways: (a) the GUM analytical approach using the law of propagation of uncertainty, and (b) Monte Carlo simulation following JCGM 101:2008 with $M = 2 \times 10^5$ trials. Use a fixed random seed so the results are reproducible.
Problem 1 (cylindricity): model $C = C_{\text{ind}} + \delta_{\text{cal}} + \delta_{\text{res}} + \delta_{\text{align}}$; five readings 8.2, 8.5, 8.1, 8.4, 8.3 μm; calibration: normal, correction 0, $U_{\text{cal}} = 0.6\,\mu$m with $k = 2$; resolution 0.1 μm, uniform, bounded by $\pm 0.05\,\mu$m; alignment: triangular centred at 0 with limits $\pm 0.30\,\mu$m.
Problem 2 (zinc concentration): $c_{Zn} = 1000\, m\, P/V$ (mg/L); purity: uniform, min 0.9985, max 1.0005; mass: Gaussian, mean 300.75 mg, SD 0.009 mg; volume due to filling: triangular, min 149.6 mL, max 150.4 mL, mean 150 mL; due to repeatability: rectangular, range 0.09 mL, mean 0; due to temperature: normal, mean 0, uncertainty 0.1 mL at 95 % confidence.
For each problem print a full GUM uncertainty budget (estimate, type, PDF, standard uncertainty, sensitivity coefficient and contribution), the combined standard uncertainty, the effective degrees of freedom (Welch–Satterthwaite), the coverage factor and the expanded uncertainty at 95 %. For the Monte Carlo part print the mean, standard uncertainty and 95 % coverage interval, and plot and save a histogram of the output PDF with the GUM Gaussian overlaid.

Prompt 2 – follow-up

Also compute the shortest 95 % coverage interval, add the validation of the GUM result against the Monte Carlo result described in JCGM 101 clause 8, and print a final summary table comparing GUM and Monte Carlo for both problems.

5.2 Verification of the generated code

The generated program was checked line by line against the hand calculation (section A) and against the independently written code. Its GUM results ($u_c = 0.33292\,\mu$m, $\nu_{\text{eff}} = 1965.4$ for Problem 1; $u_c = 2.58620$ mg/L and all sensitivity coefficients for Problem 2) agree with the hand calculation to every printed digit.

Table 8: Verification checklist for the AI-generated program.

Item checked	Finding	OK?
Measurement models	$C = C_{\text{ind}} + \sum \delta$ and $c = 1000mP/V$ implemented exactly	✓
Type A repeatability	$u = s/\sqrt{n}$ with $n-1$ in s, $\nu = 4$	✓
Normal inputs	$u_{\text{cal}} = 0.6/2$; temperature $0.1/1.96$	✓
Uniform / rectangular	$a/\sqrt{3}$; repeatability range taken as full width (± 0.045 mL)	✓
Triangular	$u = a/\sqrt{6}$; sampler arguments (min, mode, max) in the correct order	✓
Sensitivity coefficients	analytic c_m, c_P, c_V equal the hand values	✓
Welch–Satterthwaite and k	formula correct; $k = t_{0.975}(\nu_{\text{eff}})$	✓
Monte Carlo sampling	t_4 for repeatability (JCGM 101, 6.4.9); other PDFs as specified	✓
Coverage intervals	2.5 % and 97.5 % quantiles; shortest interval searched over sorted values	✓

5.3 Corrections made

No error was found in the model, the uncertainty formulas or the sampling. Four minor issues were found and corrected; none of them changes the reported results beyond Monte Carlo noise, so the values in this report are those of the program as generated.

C1. Shared random-number stream. Both problems drew from one generator, so the Problem 2 results depended on whether Problem 1 was run first. Corrected by giving each problem its own generator.

C2. Shortest-interval index. The code used $q = \lfloor pM \rfloor$ instead of the JCGM 101 (7.7.2) rule. Corrected; no effect for $M = 2 \times 10^5$, where pM is an integer.

C3. Histogram axis. The axis ran from the smallest to the largest sample, so a few extreme t_4 draws stretched the Problem 1 plot to about 11 μm (fig. 2). Corrected by limiting the axis to the central 99.9 % of the samples.

C4. Validation message. The code printed only "GUM NOT validated" using the strict two-digit tolerance. Corrected to also report the one-digit tolerance (see section 6.4).

6 Comparison and discussion

6.1 Side-by-side comparison

Table 9: Comparison of the three approaches. "k_p" is the half-width of the 95 % interval divided by the standard uncertainty.

	Method	Estimate	u	95 % interval	Half-width	k_p
P1 (μm)	GUM ($k = 1.961$)	8.3000	0.3329	[7.6471, 8.9529]	0.6529	1.961
	GUM ($k = 2$)	8.3000	0.3329	[7.6342, 8.9658]	0.6658	2.000
	Own MCM	8.2991	0.3404	[7.6335, 8.9669]	0.6667	1.959
	AI MCM	8.3002	0.3401	[7.6338, 8.9672]	0.6667	1.960
P2 (mg/L)	GUM ($k = 1.96$)	2003.998	2.586	[1998.929, 2009.066]	5.069	1.960
	GUM ($k = 2$)	2003.998	2.586	[1998.825, 2009.170]	5.172	2.000
	Own MCM	2003.986	2.586	[1998.989, 2008.994]	5.003	1.934
	AI MCM	2004.007	2.589	[1999.027, 2009.012]	4.993	1.929

6.2 Agreement

- **GUM by hand, own code and AI code give identical analytical results.** The numerical integration of the PDFs and the finite-difference sensitivity coefficients in the own code reproduce the closed-form values of the hand calculation and the AI code to 8–9 significant digits (e.g. $u(\delta_{\text{res}}) = 0.0288675133$ against the exact $0.05/\sqrt{3} = 0.0288675135\,\mu$m).
- **Estimates.** All Monte Carlo means lie within 0.001 μm (Problem 1) and 0.02 mg/L (Problem 2) of the GUM estimates. The standard error of a 2×10^5-trial mean is $u/\sqrt{M} = 0.00076\,\mu$m and 0.0058 mg/L, so these offsets are one to two standard errors, as expected from random sampling.
- **Independent vs. AI Monte Carlo.** The two independent programs, using different random-number algorithms and different seeds, agree to within this sampling noise; their histograms (figs. 1 to 4) have the same shape and the same 95 % limits to within 0.001 μm and 0.04 mg/L.

6.3 Differences and their causes

(a) Problem 1: larger standard uncertainty from MCM (0.340 vs. 0.333 μm). This is the most important difference and it is not a numerical error. Following JCGM 101, 6.4.9, the repeatability is represented by a scaled t-distribution with $\nu = n - 1 = 4$. The variance of t_ν is $\nu/(\nu - 2) = 2$, so the repeatability standard uncertainty seen by the Monte Carlo method is $\sqrt{2} \times 0.0707 = 0.100\,\mu$m, not $0.0707\,\mu$m. Substituting this value,

$$u = \sqrt{0.100^2 + 0.300^2 + 0.0289^2 + 0.1225^2} = 0.3403\,\mu\text{m},$$

which is exactly what the simulations return. GUM handles the small sample through ν_{eff} instead, but because the repeatability contributes only 4.5 % of u_c^2, $\nu_{\text{eff}} = 1965$ and k is practically 1.96. In effect GUM slightly under-states the uncertainty when n is small. The t_4 representation also explains the long tails visible in figs. 1 and 2.

(b) Problem 2: narrower MCM interval (half-width 4.99 vs. 5.07 mg/L). Here the standard uncertainties agree, but the shape of the output PDF does not. 91 % of u_c^2 comes from the

triangular filling tolerance and the uniform purity, so the central limit theorem is not fully reached. For a (linearised) sum of independent inputs the excess kurtosis of the output is $\gamma_2 = \sum_i w_i^2 \, \gamma_{2,i}$, where w_i is the variance fraction of input i and $\gamma_{2,i} = -1.2$ (rectangular), -0.6 (triangular), 0 (normal):

$$\gamma_2 \approx 0.2003^2(-1.2) + 0.7116^2(-0.6) + 0.0180^2(-1.2) = -0.35.$$

The output is therefore platykurtic: flatter at the top and lighter in the tails than a Gaussian (fig. 4). Its 95 % quantiles lie at about $\pm 1.93u$ instead of $\pm 1.96u$, so the GUM interval is slightly conservative.

(c) Coverage factor. The own code reports U with the conventional $k = 2$ ($U = 0.666\,\mu$m, 5.172 mg/L), while the AI code uses $k = t_{0.975}(\nu_{\text{eff}})$ (0.653 μm, 5.069 mg/L). For Problem 1 the cruder $k = 2$ happens to match the MCM half-width (0.667 μm) better, because it partly compensates for the effect in (a). For Problem 2, $k = 2$ over-states the interval by 3.5 %.

(d) Nonlinearity of the zinc model. $c = 1000mP/V$ is nonlinear in V. A second-order expansion predicts a bias of the MCM mean of $c\,(u_V/V)^2 = 2004 \times (0.17305/150)^2 \approx 0.0027$ mg/L, which is smaller than the standard error of the Monte Carlo mean (0.0058 mg/L). The linearisation used by GUM is therefore entirely adequate (the relative uncertainty is only 0.13 %).

(e) Monte Carlo sampling noise and random-number generators. The own code uses hand-written Box–Muller, inverse-CDF and χ^2-based samplers with seed 2026; the AI code uses NumPy's built-in samplers with seed 42. Different numbers are therefore drawn, and individual runs differ by a few standard errors of the mean.

(f) Shortest vs. symmetric intervals. Both output PDFs are almost symmetric, so the two intervals should coincide. For the flat-topped Problem 2 PDF the location of the shortest interval is poorly determined (many intervals have nearly the same width), so it shifts by a few hundredths of a mg/L between runs while its width stays the same.

(g) Numerical details. Midpoint integration ($N = 10^4$) and finite differences ($h = 10^{-6}$ forward, 10^{-4} central) introduce errors below 10^{-8} relative; they are irrelevant at the reported precision. The own histograms use 100 bins and the AI histograms 150 bins, which changes only the visual appearance.

6.4 Validation of GUM against MCM (JCGM 101, clause 8)

Table 10: End-point differences between the GUM interval (k from ν_{eff}) and the MCM intervals.

	MCM source	d_{low}	d_{high}	δ (2 digits)	δ (1 digit)
P1 (μm)	Own code	0.0136	0.0140	0.005 ✗	0.05 ✓
	AI code	0.0133	0.0143	0.005 ✗	0.05 ✓
P2 (mg/L)	Own code	0.0604	0.0722	0.05 ✗	0.5 ✓
	AI code	0.0983	0.0543	0.05 ✗	0.5 ✓

At the strict two-digit tolerance the GUM interval is not validated for either problem, for the physical reasons (a) and (b) above. If u_c is quoted to one significant digit (0.3 μm and 3 mg/L), as

is common in practice, GUM is validated in both cases. For engineering purposes the GUM results are therefore fit for use; where the second digit of the uncertainty matters, the MCM results should be preferred.

6.5 Effect of interpretation choices

Two input specifications in Problem 2 needed interpretation. If the repeatability "range 0.09 mL" were read as a half-width (±0.09 mL) instead of the full width, $u(\delta V_{\text{rep}})$ would double to 0.052 mL and u_c would rise from 2.586 to 2.655 mg/L (+2.7 %). Converting the temperature term with $k = 2$ instead of 1.96 changes u_c by less than 0.01 mg/L. Neither choice changes the conclusions, but they are the main assumptions behind the stated values.

7 Conclusions

1. The cylindricity is $C = (8.30 \pm 0.65)\,\mu$m (GUM, $k = 1.96$, 95 %); Monte Carlo gives the interval $[7.63,\ 8.97]\,\mu$m with $u = 0.34\,\mu$m. The calibration uncertainty dominates (81 % of the variance), so a better-calibrated instrument would reduce the uncertainty far more than additional repeat readings.
2. The zinc concentration is $c_{Zn} = (2004.0 \pm 5.1)$ mg/L (GUM, $k = 1.96$, 95 %); Monte Carlo gives $[1999.0,\ 2009.0]$ mg/L with $u = 2.6$ mg/L. The volume-filling tolerance (71 %) and the purity (20 %) dominate; the mass contribution is negligible.
3. The independently written code and the AI-generated code agree with each other to within Monte Carlo noise, and their GUM parts agree with the hand calculation exactly.
4. The AI-generated code was correct in its model, formulas and sampling; only four minor issues (random-number streams, shortest-interval index, plot range and validation message) needed correction. AI-generated code is useful, but it still has to be checked against an independent calculation.
5. The remaining GUM–MCM differences are explained by the t_4 representation of a five-reading repeatability (Problem 1) and by the non-Gaussian, flat-topped output PDF (Problem 2). Both effects are a few percent and are captured only by the Monte Carlo method.

References

[1] BIPM, IEC, IFCC, ILAC, ISO, IUPAC, IUPAP, OIML (2008). *Supplement 1 to the "Guide to the expression of uncertainty in measurement" – Propagation of distributions using a Monte Carlo method*, JCGM 101:2008. BIPM.

[2] Veen, A.M.H. van der, Cox, M.G. (2021). Getting started with uncertainty evaluation using the Monte Carlo method in R. *Accreditation and Quality Assurance*, 26, 129–141. https://doi.org/10.1007/s00769-021-01469-5

[3] BIPM, IEC, IFCC, ILAC, ISO, IUPAC, IUPAP, OIML (2008). *Evaluation of measurement data – Guide to the expression of uncertainty in measurement*, JCGM 100:2008. BIPM.

A Analytical solution by the GUM method

This appendix works both problems analytically, step by step.

Standard uncertainty of the rectangular and triangular PDFs

Rectangular on $[-a, a]$: $g(x) = \dfrac{1}{2a}$, so

$$u^2 = \int_{-a}^{a} x^2 \, \frac{1}{2a} \, dx = \frac{1}{2a}\left[\frac{x^3}{3}\right]_{-a}^{a} = \frac{a^2}{3} \quad \Longrightarrow \quad u = \frac{a}{\sqrt{3}}.$$

Symmetric triangular on $[-a, a]$: $g(x) = \dfrac{a - |x|}{a^2}$, so

$$u^2 = 2\int_{0}^{a} x^2 \, \frac{a - x}{a^2} \, dx = \frac{2}{a^2}\left[\frac{ax^3}{3} - \frac{x^4}{4}\right]_{0}^{a} = \frac{2}{a^2} \cdot \frac{a^4}{12} = \frac{a^2}{6} \quad \Longrightarrow \quad u = \frac{a}{\sqrt{6}}.$$

Normal with expanded uncertainty U at coverage factor k: $u = U/k$.

A.1 Problem 1: Cylindricity

Step 1 – Measurement model

$$C = C_{\text{ind}} + \delta_{\text{cal}} + \delta_{\text{res}} + \delta_{\text{align}}, \qquad \delta_{\text{cal}} = \delta_{\text{res}} = \delta_{\text{align}} = 0 \;\Rightarrow\; C = C_{\text{ind}}.$$

Step 2 – Mean and Type A uncertainty of the readings

j	C_j (μm)	$C_j - \bar{C}$ (μm)	$(C_j - \bar{C})^2$ (μm^2)
1	8.2	−0.1	0.01
2	8.5	+0.2	0.04
3	8.1	−0.2	0.04
4	8.4	+0.1	0.01
5	8.3	0.0	0.00
Σ	41.5	0.0	0.10

$$\bar{C}_{\text{ind}} = \frac{41.5}{5} = 8.30\,\mu\text{m}$$

$$s^2 = \frac{\sum (C_j - \bar{C})^2}{n-1} = \frac{0.10}{4} = 0.025\,\mu\text{m}^2, \qquad s = \sqrt{0.025} = 0.15811\,\mu\text{m}$$

$$u(C_{\text{ind}}) = \frac{s}{\sqrt{n}} = \frac{0.15811}{\sqrt{5}} = \frac{0.15811}{2.23607} = 0.07071\,\mu\text{m}, \qquad \nu_A = n - 1 = 4$$

Step 3 – Type B standard uncertainties

$$\text{Calibration (normal):} \quad u(\delta_{\text{cal}}) = \frac{U_{\text{cal}}}{k} = \frac{0.6}{2} = 0.30000\,\mu\text{m}$$

$$\text{Resolution (rectangular, } a = 0.05\text{):} \quad u(\delta_{\text{res}}) = \frac{0.05}{\sqrt{3}} = \frac{0.05}{1.73205} = 0.02887\,\mu\text{m}$$

$$\text{Alignment (triangular, } a = 0.30\text{):} \quad u(\delta_{\text{align}}) = \frac{0.30}{\sqrt{6}} = \frac{0.30}{2.44949} = 0.12247\,\mu\text{m}$$

All three are Type B with $\nu = \infty$.

Step 4 – Sensitivity coefficients

$$c_{\text{ind}} = \frac{\partial C}{\partial C_{\text{ind}}} = 1, \quad c_{\text{cal}} = \frac{\partial C}{\partial \delta_{\text{cal}}} = 1, \quad c_{\text{res}} = \frac{\partial C}{\partial \delta_{\text{res}}} = 1, \quad c_{\text{align}} = \frac{\partial C}{\partial \delta_{\text{align}}} = 1.$$

Step 5 – Combined standard uncertainty

Source	$u(x_i)$ (μm)	c_i	$(c_i u(x_i))^2$ (μm^2)	%
Repeatability	0.07071	1	0.005000	4.51
Calibration	0.30000	1	0.090000	81.20
Resolution	0.02887	1	0.000833	0.75
Alignment	0.12247	1	0.015000	13.53
Sum			0.110833	100

$$u_c(C) = \sqrt{0.110833} = 0.33292\,\mu\text{m}$$

Step 6 – Effective degrees of freedom (Welch–Satterthwaite)

Only the repeatability has finite degrees of freedom, so with (5):

$$\nu_{\text{eff}} = \frac{u_c^4}{\dfrac{(c_{\text{ind}}\,u(C_{\text{ind}}))^4}{\nu_A}} = \frac{(0.110833)^2}{\dfrac{(0.005000)^2}{4}} = \frac{0.012284}{6.25 \times 10^{-6}} = 1965.4$$

Step 7 – Coverage factor and expanded uncertainty (95 %)

From the t-table, for $\nu_{\text{eff}} = 1965$ (beyond the $\nu = 1000$ row) the two-sided 95 % value is $t_{0.975} = 1.961 \approx 1.96$.

$$U = k\,u_c = 1.961 \times 0.33292 = 0.6529\,\mu\text{m} \approx 0.65\,\mu\text{m}$$

$$(\text{with } k = 2: \quad U = 2 \times 0.33292 = 0.6658\,\mu\text{m} \approx 0.67\,\mu\text{m})$$

$$95\,\%\text{ interval: } [8.30 - 0.653,\ 8.30 + 0.653] = [7.647,\ 8.953]\,\mu\text{m}$$

Result (Problem 1): $C = (8.30 \pm 0.65)\,\mu\text{m}$, $u_c = 0.33\,\mu\text{m}$, $k = 1.96$, $\nu_{\text{eff}} \approx 1965$, coverage probability 95 %.

A.2 Problem 2: Zinc concentration

Step 1 – Measurement model and best estimates

$$c_{Zn} = \frac{1000\, m\, P}{V}, \qquad V = V_{\text{fill}} + \delta V_{\text{rep}} + \delta V_{\text{temp}}$$

$$P_0 = \frac{0.9985 + 1.0005}{2} = 0.9995 \quad \text{(centre of the uniform PDF)}$$
$$m_0 = 300.75\,\text{mg}$$
$$V_0 = 150 + 0 + 0 = 150\,\text{mL}$$

Step 2 – Estimate of the concentration

$$c_{Zn} = \frac{1000 \times 300.75 \times 0.9995}{150} = \frac{300\,599.625}{150} = 2003.9975\,\text{mg/L}$$

Step 3 – Standard uncertainties of the inputs

$$\text{Purity (uniform, } a = \tfrac{1.0005-0.9985}{2} = 0.001\text{):} \quad u(P) = \frac{0.001}{\sqrt{3}} = 5.7735 \times 10^{-4}$$
$$\text{Mass (Gaussian):} \quad u(m) = 0.009\,\text{mg}$$
$$\text{Filling (triangular, } a = \tfrac{150.4-149.6}{2} = 0.4\text{):} \quad u(V_{\text{fill}}) = \frac{0.4}{\sqrt{6}} = 0.16330\,\text{mL}$$
$$\text{Repeatability (rectangular, range } 0.09 \Rightarrow a = 0.045\text{):} \quad u(\delta V_{\text{rep}}) = \frac{0.045}{\sqrt{3}} = 0.025981\,\text{mL}$$
$$\text{Temperature (normal, 0.1 mL at 95 \%):} \quad u(\delta V_{\text{temp}}) = \frac{0.1}{1.96} = 0.051020\,\text{mL}$$

Step 4 – Combined uncertainty of the volume

The three volume components add linearly ($\partial V/\partial V_k = 1$), so

$$\begin{aligned} u^2(V) &= 0.16330^2 + 0.025981^2 + 0.051020^2 \\ &= 0.026667 + 0.000675 + 0.002603 = 0.029945\,\text{mL}^2, \\ u(V) &= \sqrt{0.029945} = 0.17305\,\text{mL} \end{aligned}$$

Step 5 – Sensitivity coefficients

Partial derivatives of $c = 1000mP/V$, evaluated at the best estimates:

$$c_m = \frac{\partial c}{\partial m} = \frac{1000\,P}{V} = \frac{1000 \times 0.9995}{150} = 6.6633\ \mathrm{L^{-1}}$$
$$c_P = \frac{\partial c}{\partial P} = \frac{1000\,m}{V} = \frac{1000 \times 300.75}{150} = 2005.0\,\mathrm{mg/L}$$
$$c_V = \frac{\partial c}{\partial V} = -\frac{1000\,m\,P}{V^2} = -\frac{c_{Zn}}{V} = -\frac{2003.9975}{150} = -13.360\ \mathrm{mg\,L^{-1}\,mL^{-1}}$$

The same c_V applies to each of the three volume components.

Step 6 – Combined standard uncertainty

Source	$u(x_i)$	c_i	$\lvert c_i \rvert u(x_i)$ (mg/L)	$(c_i u(x_i))^2$	%
Purity P	5.7735×10^{-4}	2005.0	1.1576	1.3400	20.03
Mass m	0.009 mg	6.6633	0.0600	0.0036	0.05
V_{fill}	0.16330 mL	−13.360	2.1817	4.7597	71.16
δV_{rep}	0.025981 mL	−13.360	0.3471	0.1205	1.80
δV_{temp}	0.051020 mL	−13.360	0.6816	0.4646	6.95
Sum				6.6884	100

$$u_c(c_{Zn}) = \sqrt{6.6884} = 2.5862\,\mathrm{mg/L}$$

Check with the relative form (4):

$$\frac{u(P)}{P} = \frac{5.7735 \times 10^{-4}}{0.9995} = 5.7764 \times 10^{-4}, \qquad \frac{u(m)}{m} = \frac{0.009}{300.75} = 2.9925 \times 10^{-5},$$
$$\frac{u(V)}{V} = \frac{0.17305}{150} = 1.1536 \times 10^{-3},$$
$$\frac{u_c}{c} = \sqrt{(5.7764 \times 10^{-4})^2 + (2.9925 \times 10^{-5})^2 + (1.1536 \times 10^{-3})^2}$$
$$= 1.2905 \times 10^{-3}\ (0.129\,\%),$$
$$u_c = 1.2905 \times 10^{-3} \times 2003.9975 = 2.5862\,\mathrm{mg/L} \quad 3$$

Step 7 – Effective degrees of freedom, coverage factor and expanded uncertainty

No input is given with a finite number of degrees of freedom (the repeatability is specified directly as a rectangular PDF, without a number of observations), so $\nu_{\mathrm{eff}} = \infty$ and $k = 1.96$ for 95 %:

$$U = 1.96 \times 2.5862 = 5.069\,\mathrm{mg/L} \approx 5.1\,\mathrm{mg/L}$$
$$(\text{with } k = 2: \quad U = 2 \times 2.5862 = 5.172\,\mathrm{mg/L})$$

$$95\,\%\ \text{interval: } [2003.998 - 5.069,\ 2003.998 + 5.069] = [1998.929,\ 2009.066]\,\mathrm{mg/L}$$

Result (Problem 2): $c_{Zn} = (2004.0 \pm 5.1)\,\mathrm{mg/L}$, $u_c = 2.6\,\mathrm{mg/L}$ (0.13 % relative), $k = 1.96$, coverage probability 95 %.

B Raw program output

The console output of each program is reproduced unedited below.

B.1 Independent code: Problem 1 (uncertainty_problem_1.py)

```
PROBLEM 1 - GUM
mean = 8.3
s = 0.15811388300841925
u_A = 0.07071067811865488
u_cal = 0.3
u_res = 0.028867513315143808
u_align = 0.12247448744534521
uc = 0.33291640602409495
nu_eff = 1965.4444468088766
k95 = 1.9611717052452833
U(k = 2) = 0.6658328120481899
U(k95) = 0.6529062357064054


PROBLEM 1 - MONTE CARLO
mean = 8.299113114354606
u = 0.3403970634700122
95% low = 7.6334606598101
95% high = 8.966934905744893
```

B.2 Independent code: Problem 2 (uncertainty_problem_2.py)

```
PROBLEM 2 - GUM
c_Zn (mg/L) = 2003.9975
u(P) = 0.0005773502663028423
u(m) (mg) = 0.009
u(V) (mL) = 0.1730455108992919
c_m = 6.663333331289323
c_P = 2004.9999999991996
c_V = -13.359983333884884
u_c (mg/L) = 2.5861975231726846
U, k=2 (mg/L) = 5.172395046345369


PROBLEM 2 - MONTE CARLO
mean = 2003.9863393154894
u = 2.58610954649416
95% low = 1998.9890461326738
95% high = 2008.9941381355193
```

B.3 AI-generated code (uncertainty_analysis.py)

```


================================================================================================
GUM UNCERTAINTY BUDGET - PROBLEM 1 (Cylindricity)
================================================================================================
Source                        Estimate  Type          PDF       u(xi)          ci    |ci|u(xi)
------------------------------------------------------------------------------------------------
Repeatability (C_ind)              8.3     A     t (nu=4)  7.0711e-02  1.0000e+00  7.0711e-02
Calibration d_cal                    0     B       Normal  3.0000e-01  1.0000e+00  3.0000e-01
Resolution d_res                     0     B      Uniform  2.8868e-02  1.0000e+00  2.8868e-02
Alignment d_align                    0     B   Triangular  1.2247e-01  1.0000e+00  1.2247e-01
------------------------------------------------------------------------------------------------
Estimate of output y            = 8.30000 um
Combined standard uncertainty uc = 0.33292 um
Effective degrees of freedom     = 1965.4
Coverage factor k (95 %)         = 1.961
```

```
Expanded uncertainty U           = 0.65291 um
95 % interval (GUM)              = [7.64709, 8.95291] um

------------------------------------------------------------------------------------------
MONTE CARLO RESULTS - PROBLEM 1 (Cylindricity)  (M = 200,000 trials)
------------------------------------------------------------------------------------------
Mean                              = 8.30025 um
Standard uncertainty (std. dev.)  = 0.34014 um
95 % symmetric interval           = [7.63375, 8.96721] um
95 % shortest interval            = [7.64015, 8.97317] um
Half-width of symmetric interval  = 0.66673 um

GUM validation: delta = 5.00e-03, d_low = 1.33e-02, d_high = 1.43e-02  ->  GUM NOT validated at this
    tolerance
Histogram saved as problem1_cylindricity_histogram.png

==========================================================================================
GUM UNCERTAINTY BUDGET - PROBLEM 2 (Zinc concentration)
==========================================================================================
Source                         Estimate  Type          PDF       u(xi)          ci   |ci|u(xi)
------------------------------------------------------------------------------------------
Purity P                         0.9995     B      Uniform  5.7735e-04  2.0050e+03  1.1576e+00
Mass m (mg)                      300.75     B     Gaussian  9.0000e-03  6.6633e+00  5.9970e-02
Volume - filling (mL)               150     B   Triangular  1.6330e-01 -1.3360e+01  2.1817e+00
Volume - repeat. (mL)                 0     A  Rectangular  2.5981e-02 -1.3360e+01  3.4710e-01
Volume - temp. (mL)                   0     B       Normal  5.1021e-02 -1.3360e+01  6.8164e-01
------------------------------------------------------------------------------------------
Estimate of output y            = 2003.99750 mg/L
Combined standard uncertainty uc = 2.58620 mg/L
Effective degrees of freedom     = infinite
Coverage factor k (95 %)         = 1.960
Expanded uncertainty U           = 5.06886 mg/L
95 % interval (GUM)              = [1998.92864, 2009.06636] mg/L
(Combined u(V) = 0.17305 mL, relative uc = 0.1291 %)

------------------------------------------------------------------------------------------
MONTE CARLO RESULTS - PROBLEM 2 (Zinc concentration)  (M = 200,000 trials)
------------------------------------------------------------------------------------------
Mean                              = 2004.00657 mg/L
Standard uncertainty (std. dev.)  = 2.58861 mg/L
95 % symmetric interval           = [1999.02684, 2009.01215] mg/L
95 % shortest interval            = [1999.04909, 2009.03077] mg/L
Half-width of symmetric interval  = 4.99266 mg/L

GUM validation: delta = 5.00e-02, d_low = 9.82e-02, d_high = 5.42e-02  ->  GUM NOT validated at this
    tolerance
Histogram saved as problem2_zinc_histogram.png

==========================================================================================
SUMMARY: GUM vs MONTE CARLO
==========================================================================================

Problem 1 - Cylindricity
                 Estimate     Std. unc.                  95 % interval
GUM                8.3000        0.3329               [7.6471, 8.9529]
Monte Carlo        8.3002        0.3401               [7.6337, 8.9672]  (um)

Problem 2 - Zinc concentration
                 Estimate     Std. unc.                  95 % interval
GUM              2003.9975        2.5862         [1998.9286, 2009.0664]
Monte Carlo      2004.0066        2.5886         [1999.0268, 2009.0122]  (mg/L)
```

C Source code

The complete source code of every program used in this report is reproduced below.

C.1 Independent code: Problem 1 (uncertainty_problem_1.py)

```python
import numpy as np
import matplotlib.pyplot as plt
import math
from scipy import stats

x = np.array([8.2, 8.5, 8.1, 8.4, 8.3])
n = len(x)
x_bar = np.mean(x)
C_ind = x_bar

dev = x - x_bar
dev_sq = dev**2

sum_dev = np.sum(dev) # sum of deviations
sum_dev_sq = np.sum(dev_sq) # sum of squares

s_sq = sum_dev_sq / (n - 1)
s = math.sqrt(s_sq)

# type A uncertainty

u2_A = s_sq / n
u_A = math.sqrt(u2_A)
v = n - 1

# type B uncertainty

# calibration - normal, U = 0.6, k = 2

U = 0.6
k = 2

del_cal = U / k
u2_del_cal = del_cal**2

# resolution - uniform, a = +-0.05 --> numeric integration

def uniform(a, N):
    dx = (2 * a) / N

    area = 0.0
    u2_del = 0.0

    def pdf(x):
        return 1 / (2 * a)

    for i in range(N):
        x = -a + (i + 0.5) * dx
        area += pdf(x) * dx
        u2_del += x**2 * pdf(x) * dx

    u_del = math.sqrt(u2_del)

    return u_del, u2_del

uni_tup = uniform(0.05, 10000)
u_del_res = uni_tup[0]
u2_del_res = uni_tup[1]


# alignment - triangular, a = +-0.3 --> numeric integration

```

```python
def triangular(a, N):
    dx = a / N

    def pdf(x):
        return (a - x) / a**2

    area = 0.0
    u2_del = 0.0
    for i in range(N):
        x = (i + 0.5) * dx
        area += 2 * pdf(x) * dx
        u2_del += 2 * x**2 * pdf(x) * dx

    u_del = math.sqrt(u2_del)

    return u_del, u2_del

tri_tup = triangular(0.3, 10000)
u_del_align = tri_tup[0]
u2_del_align = tri_tup[1]

# sensitivity coefficient

def model(c_ind, d_cal, d_res, d_align):
    return c_ind + d_cal + d_res + d_align

h = 1e-6                          # tiny nudge
ref = [0.0, 0.0, 0.0, 0.0]        # reference point (any values work, see note)

def sensitivity(i):
    nudged = ref.copy()
    nudged[i] += h
    return (model(*nudged) - model(*ref)) / h

c_A     = sensitivity(0)
c_cal   = sensitivity(1)
c_res   = sensitivity(2)
c_align = sensitivity(3)

# combined standard uncertainty

term_A     = c_A**2     * u2_A
term_cal   = c_cal**2   * u2_del_cal
term_res   = c_res**2   * u2_del_res
term_align = c_align**2 * u2_del_align

u2_c = term_A + term_cal + term_res + term_align
u_c  = math.sqrt(u2_c)

# effective degrees of freedom

nu_cal   = math.inf          # Type B → infinite degrees of freedom
nu_res   = math.inf
nu_align = math.inf

denominator = (term_A**2     / v        +
               term_cal**2   / nu_cal   +
               term_res**2   / nu_res   +
               term_align**2 / nu_align)

nu_eff = u_c**4 / denominator

# coverage factor and expanded uncertainty

t95   = stats.t.ppf(0.975, nu_eff)   # t-table value for 95 %, ν = ν_eff
k_cov = 2                            # conventional choice
U_exp = k_cov * u_c                  # expanded uncertainty, k = 2
U_196 = 1.96 * u_c                   # alternative, k = 1.96
U_rep = round(U_exp, 2)              # rounded: 0.67

# final result and interval

```

```
C_result = C_ind
lower = C_result - U_rep
upper = C_result + U_rep

print("PROBLEM 1 - GUM")
print("mean =", C_ind)
print("s =", s)
print("u_A =", u_A)
print("u_cal =", del_cal)
print("u_res =", u_del_res)
print("u_align =", u_del_align)
print("uc =", u_c)
print("nu_eff =", nu_eff)
print("k95 =", t95)
print("U(k = 2) =", U_exp)
print("U(k95) =", t95 * u_c)


# monte carlo simulation

M = 200000
rng = np.random.default_rng(2026)

# samplers

def sample_uniform(a, b, size):
    U = rng.uniform(0, 1, size)
    return a + (b - a) * U

def sample_triangular(a, c, b, size):        # a = min, c = peak, b = max
    U = rng.uniform(0, 1, size)
    F = (c - a) / (b - a)
    left  = a + np.sqrt(U * (b - a) * (c - a))
    right = b - np.sqrt((1 - U) * (b - a) * (b - c))
    return np.where(U < F, left, right)

def sample_normal(mu, sigma, size):
    U1 = 1 - rng.uniform(0, 1, size)
    U2 = rng.uniform(0, 1, size)
    z = np.sqrt(-2 * np.log(U1)) * np.cos(2 * np.pi * U2)
    return mu + sigma * z

def sample_t(nu, size):
    z = sample_normal(0, 1, size)
    chi2 = np.zeros(size)
    for j in range(nu):
        chi2 += sample_normal(0, 1, size)**2
    return z / np.sqrt(chi2 / nu)

C_ind_mc   = C_ind + (s / math.sqrt(n)) * sample_t(v, M)   # type A: t with ν = 4
d_cal_mc   = sample_normal(0, del_cal, M)                  # normal, mean 0, σ = 0.3
d_res_mc   = sample_uniform(-0.05, 0.05, M)                # uniform +-0.05
d_align_mc = sample_triangular(-0.3, 0, 0.3, M)            # triangular +-0.3, peak at 0

C_mc = model(C_ind_mc, d_cal_mc, d_res_mc, d_align_mc)

# plotting histogram

mean_mc = np.mean(C_mc)               # best estimate
u_mc    = np.std(C_mc, ddof=1)        # standard uncertainty
low_mc  = np.percentile(C_mc, 2.5)    # lower end of 95 % interval
high_mc = np.percentile(C_mc, 97.5)   # upper end of 95 % interval

plt.figure(figsize=(8, 5))
plt.hist(C_mc, bins=100, density=True, alpha=0.6, label="Monte Carlo")
plt.axvline(low_mc, color="k", linestyle="--", label="95% interval")
plt.axvline(high_mc, color="k", linestyle="--")
plt.xlabel("Cylindricity C (micro-meter)")
plt.ylabel("Probability density")
plt.title("Problem 1: Monte Carlo PDF")
plt.legend()

```

```
print('\n')

print("PROBLEM 1 - MONTE CARLO")
print("mean =", mean_mc)
print("u =", u_mc)
print("95% low =", low_mc)
print("95% high =", high_mc)

plt.show()
```

C.2 Independent code: Problem 2 (uncertainty_problem_2.py)

```
import numpy as np
import math
import matplotlib.pyplot as plt

def model(m, P, V):
    C_zn = (1000 * m * P) / V
    return C_zn

# best estimates

P = np.array([0.9985, 1.0005])
V = np.array([149.6, 150.4])
P_not = np.mean(P)
m_not = 300.75
V_not = 150
N = 10000

C_zn_not = model(m_not, P_not, V_not)

# type B uncertainties

# purity --> uniform (max, min)

def uniform(x_min, x_max, N):
    a_x = (x_max - x_min) / 2
    dx = (2 * a_x) / N

    def pdf(x):
        return 1 / (2 * a_x)

    area_x = 0.0
    u2_x = 0.0
    for i in range(N):
        x = -a_x + (i + 0.5) * dx
        area_x += pdf(x) * dx
        u2_x += x**2 * pdf(x) * dx

    u_x = math.sqrt(u2_x)

    return u_x, u2_x

P_tuple = uniform(P[0], P[1], N)
u_P = P_tuple[0]
u2_P = P_tuple[1]

# mass --> gaussian(mean, SD)

m_sd = 0.009
u_m = 0.009
u2_m = 0.009**2

# volume --> triangular, repeatability, temperature

# filling --> triangular (min, max, min)

def triangular(x_max, x_min, N):
    a_x = (x_max - x_min) / 2
    dx = a_x / N

```

```
    def pdf(x):
        return (a_x - x) / a_x**2

    area_x = 0.0
    u2_x = 0.0
    for i in range(N):
        x = (i + 0.5) * dx
        area_x += 2 * pdf(x) * dx
        u2_x += 2 * x**2 * pdf(x) * dx

    u_x = math.sqrt(u2_x)
    return u_x, u2_x

V_tuple_fill = triangular(V[1], V[0], N)
u_V_fill = V_tuple_fill[0]
u2_V_fill = V_tuple_fill[1]

# repeatability --> (range, mean)

def rectangular(range_, N):
    x_rep = range_ / 2
    dx = (2 * x_rep) / N

    def pdf(x):
        return 1 / (2 * x_rep)

    area_x = 0.0
    u2_x = 0.0

    for i in range(N):
        x = -x_rep + (i + 0.5) * dx
        area_x += pdf(x) * dx
        u2_x += x**2 * pdf(x) * dx

    u_x = math.sqrt(u2_x)
    return u_x, u2_x

range_repeat = 0.09
V_tuple_repeat = rectangular(range_repeat, N)
u_V_repeat = V_tuple_repeat[0]
u2_V_repeat = V_tuple_repeat[1]

# temperature --> normal, at 95%, k = 1.96

u_temp = 0.1
u_V_temp = u_temp / 1.96
u2_V_temp = u_V_temp**2

# combined uncertainty

u2_V = u2_V_fill + u2_V_repeat + u2_V_temp
u_V = math.sqrt(u2_V)

# sensitivity coefficients

def sensitivity(f, inputs, i, h = 1e-4):
    up = list(inputs)
    down = list(inputs)
    up[i] += h
    down[i] -= h
    return (f(*up) - f(*down)) / (2 * h)

inputs = [m_not, P_not, V_not]

c_m = sensitivity(model, inputs, 0)
c_P = sensitivity(model, inputs, 1)
c_V = sensitivity(model, inputs, 2)

# individual contribution

m_contri = c_m * u_m
P_contri = c_P * u_P
```

```python
V_contri = c_V * u_V
m2_contri = u2_m * c_m**2
P2_contri = u2_P * c_P**2
V2_contri = u2_V * c_V**2

# combined standard uncertainty

u2_combined = m2_contri + P2_contri + V2_contri
u_combined = math.sqrt(u2_combined)

print("PROBLEM 2 - GUM")
print("c_Zn (mg/L) =", C_zn_not)
print("u(P) =", u_P)
print("u(m) (mg) =", u_m)
print("u(V) (mL) =", u_V)
print("c_m =", c_m)
print("c_P =", c_P)
print("c_V =", c_V)
print("u_c (mg/L) =", u_combined)
print("U, k=2 (mg/L) =", 2 * u_combined)

# monte carlo simulation

M = 200000
rng = np.random.default_rng(2026)

# samplers

def sample_uniform(a, b, size):
    U = rng.uniform(0, 1, size)
    return a + (b - a) * U

def sample_triangular(a, c, b, size):        # a = min, c = peak, b = max
    U = rng.uniform(0, 1, size)
    F = (c - a) / (b - a)
    left  = a + np.sqrt(U * (b - a) * (c - a))
    right = b - np.sqrt((1 - U) * (b - a) * (b - c))
    return np.where(U < F, left, right)

def sample_normal(mu, sigma, size):
    U1 = 1 - rng.uniform(0, 1, size)
    U2 = rng.uniform(0, 1, size)
    z = np.sqrt(-2 * np.log(U1)) * np.cos(2 * np.pi * U2)
    return mu + sigma * z

P_mc      = sample_uniform(P[0], P[1], M)            # purity: uniform
m_mc      = sample_normal(m_not, u_m, M)                    # mass: Gaussian
V_fill_mc = sample_triangular(V[0], V_not, V[1], M)    # filling: triangular
V_rep_mc = sample_uniform(-range_repeat / 2, range_repeat / 2, M)            # repeatability:
    ↪ rectangular
V_temp_mc = sample_normal(0, u_V_temp, M)                  # temperature: normal

V_mc = V_fill_mc + V_rep_mc + V_temp_mc
c_mc = model(m_mc, P_mc, V_mc)

mean_mc = np.mean(c_mc)
u_mc    = np.std(c_mc, ddof=1)
low_mc  = np.percentile(c_mc, 2.5)
high_mc = np.percentile(c_mc, 97.5)

plt.figure(figsize=(8, 5))
plt.hist(c_mc, bins=100, density=True, alpha=0.6, label="Monte Carlo")

# xs = np.linspace(c_mc.min(), c_mc.max(), 500)
# gum_pdf = (1 / (u_combined * math.sqrt(2 * math.pi))) * np.exp(-0.5 * ((xs - C_zn_not) / u_combined)**2)
    ↪
# plt.plot(xs, gum_pdf, "r-", label="GUM (normal)")

plt.axvline(low_mc, color="k", linestyle="--", label="95% interval")
plt.axvline(high_mc, color="k", linestyle="--")
# plt.axvline(mean_mc, color="g", label="MC mean")
plt.xlabel("Zinc concentration c_Zn (mg/L)")
```

```
plt.ylabel("Probability density")
plt.title("Problem 2: Monte Carlo PDF")
plt.legend()

print('\n')

print("PROBLEM 2 - MONTE CARLO")
print("mean =", mean_mc)
print("u =", u_mc)
print("95% low =", low_mc)
print("95% high =", high_mc)

plt.show()
```

C.3 AI-generated code (uncertainty_analysis.py)

```
"""
ME3112 Metrology and Instrumentation - Uncertainty Analysis Assignment
Problem 1: Cylindricity measurement
Problem 2: Zinc concentration

Each problem is solved two ways:
  (a) GUM analytical approach (law of propagation of uncertainty)
  (b) Monte Carlo simulation (JCGM 101:2008), M = 2e5 trials

Requires: numpy, scipy, matplotlib
"""

import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

M = 200_000            # number of Monte Carlo trials
SEED = 42              # fixed seed so results are reproducible
rng = np.random.default_rng(SEED)


# ------------------------------------------------------------------------
# Helper functions
# ------------------------------------------------------------------------
def shortest_interval(samples, p=0.95):
    """Shortest coverage interval (JCGM 101, clause 7.7)."""
    y = np.sort(samples)
    n = len(y)
    q = int(np.floor(p * n))
    widths = y[q:] - y[:n - q]
    i = np.argmin(widths)
    return y[i], y[i + q]


def symmetric_interval(samples, p=0.95):
    """Probabilistically symmetric coverage interval (2.5 % and 97.5 % quantiles)."""
    return np.percentile(samples, [100 * (1 - p) / 2, 100 * (1 + p) / 2])


def welch_satterthwaite(uc, contributions, dofs):
    """Effective degrees of freedom. contributions = |c_i|*u_i, dofs = nu_i (np.inf for Type B)."""
    denom = sum((ci ** 4) / nu for ci, nu in zip(contributions, dofs) if np.isfinite(nu))
    return np.inf if denom == 0 else uc ** 4 / denom


def print_budget(title, rows, y, uc, nu_eff, k, unit):
    """rows = list of (name, estimate, type, distribution, u_i, c_i)."""
    print("\n" + "=" * 92)
    print(f"GUM UNCERTAINTY BUDGET - {title}")
    print("=" * 92)
    print(f"{'Source':<26}{'Estimate':>12}{'Type':>6}{'PDF':>13}"
          f"{'u(xi)':>12}{'ci':>12}{'|ci|u(xi)':>12}")
    print("-" * 92)
    for name, est, typ, pdf, u, c in rows:
        print(f"{name:<26}{est:>12.5g}{typ:>6}{pdf:>13}{u:>12.4e}{c:>12.4e}{abs(c) * u:>12.4e}")
```

```
    print("-" * 92)
    U = k * uc
    print(f"Estimate of output y            = {y:.5f} {unit}")
    print(f"Combined standard uncertainty uc = {uc:.5f} {unit}")
    print(f"Effective degrees of freedom     = {nu_eff:.1f}" if np.isfinite(nu_eff)
          else "Effective degrees of freedom     = infinite")
    print(f"Coverage factor k (95 %)         = {k:.3f}")
    print(f"Expanded uncertainty U           = {U:.5f} {unit}")
    print(f"95 % interval (GUM)              = [{y - U:.5f}, {y + U:.5f}] {unit}")
    return U


def print_mc(title, samples, unit):
    mean = samples.mean()
    sd = samples.std(ddof=1)
    lo_s, hi_s = symmetric_interval(samples)
    lo_sh, hi_sh = shortest_interval(samples)
    print("\n" + "-" * 92)
    print(f"MONTE CARLO RESULTS - {title}  (M = {len(samples):,} trials)")
    print("-" * 92)
    print(f"Mean                                = {mean:.5f} {unit}")
    print(f"Standard uncertainty (std. dev.)  = {sd:.5f} {unit}")
    print(f"95 % symmetric interval           = [{lo_s:.5f}, {hi_s:.5f}] {unit}")
    print(f"95 % shortest interval            = [{lo_sh:.5f}, {hi_sh:.5f}] {unit}")
    print(f"Half-width of symmetric interval  = {(hi_s - lo_s) / 2:.5f} {unit}")
    return mean, sd, (lo_s, hi_s)


def validate_gum(y, U, mc_interval, uc, ndig=2):
    """GUM validation against MCM (JCGM 101, clause 8).
    Numerical tolerance delta = 0.5 * 10^l, where uc is written with ndig significant digits."""
    l = int(np.floor(np.log10(uc))) - (ndig - 1)
    delta = 0.5 * 10 ** l
    d_low = abs((y - U) - mc_interval[0])
    d_high = abs((y + U) - mc_interval[1])
    ok = d_low <= delta and d_high <= delta
    print(f"\nGUM validation: delta = {delta:.2e}, d_low = {d_low:.2e}, d_high = {d_high:.2e}"
          f"  ->  {'GUM VALIDATED' if ok else 'GUM NOT validated'} at this tolerance")


def plot_histogram(samples, y_gum, uc_gum, interval, xlabel, title, fname):
    fig, ax = plt.subplots(figsize=(9, 5.5))
    ax.hist(samples, bins=150, density=True, color="steelblue", alpha=0.65,
            edgecolor="none", label="Monte Carlo PDF")
    x = np.linspace(samples.min(), samples.max(), 600)
    ax.plot(x, stats.norm.pdf(x, y_gum, uc_gum), "r-", lw=2,
            label="GUM (Gaussian, y, $u_c$)")
    ax.axvline(samples.mean(), color="k", ls="-", lw=1.5,
               label=f"MC mean = {samples.mean():.4f}")
    ax.axvline(interval[0], color="green", ls="--", lw=1.5,
               label=f"95 % interval [{interval[0]:.4f}, {interval[1]:.4f}]")
    ax.axvline(interval[1], color="green", ls="--", lw=1.5)
    ax.set_xlabel(xlabel)
    ax.set_ylabel("Probability density")
    ax.set_title(title)
    ax.legend(fontsize=9)
    ax.grid(alpha=0.3)
    fig.tight_layout()
    fig.savefig(fname, dpi=200)
    print(f"Histogram saved as {fname}")


# ======================================================================
# PROBLEM 1 : CYLINDRICITY   C = C_ind + d_cal + d_res + d_align
# ======================================================================
def problem1():
    readings = np.array([8.2, 8.5, 8.1, 8.4, 8.3])           # um
    n = len(readings)
    C_ind = readings.mean()
    s = readings.std(ddof=1)
    u_rep = s / np.sqrt(n)                                    # Type A
    nu_rep = n - 1
```

```
    U_cal, k_cal = 0.6, 2.0
    u_cal = U_cal / k_cal                                   # Normal
    a_res = 0.05
    u_res = a_res / np.sqrt(3)                              # Uniform, +/-0.05 um
    a_align = 0.30
    u_align = a_align / np.sqrt(6)                          # Symmetric triangular, +/-0.30 um

    # ---- GUM: model is linear, all sensitivity coefficients = 1 ----
    c = [1.0, 1.0, 1.0, 1.0]
    u = [u_rep, u_cal, u_res, u_align]
    dofs = [nu_rep, np.inf, np.inf, np.inf]
    C = C_ind + 0 + 0 + 0
    uc = np.sqrt(sum((ci * ui) ** 2 for ci, ui in zip(c, u)))
    nu_eff = welch_satterthwaite(uc, [ci * ui for ci, ui in zip(c, u)], dofs)
    k = stats.t.ppf(0.975, nu_eff) if np.isfinite(nu_eff) else stats.norm.ppf(0.975)

    rows = [("Repeatability (C_ind)", C_ind, "A", "t (nu=4)", u_rep, c[0]),
            ("Calibration d_cal", 0.0, "B", "Normal", u_cal, c[1]),
            ("Resolution d_res", 0.0, "B", "Uniform", u_res, c[2]),
            ("Alignment d_align", 0.0, "B", "Triangular", u_align, c[3])]
    U = print_budget("PROBLEM 1 (Cylindricity)", rows, C, uc, nu_eff, k, "um")

    # ---- Monte Carlo ----
    # Repeatability: scaled & shifted t-distribution with n-1 dof (JCGM 101, 6.4.9)
    C_ind_s = C_ind + u_rep * rng.standard_t(nu_rep, M)
    d_cal_s = rng.normal(0.0, u_cal, M)
    d_res_s = rng.uniform(-a_res, a_res, M)
    d_align_s = rng.triangular(-a_align, 0.0, a_align, M)
    C_s = C_ind_s + d_cal_s + d_res_s + d_align_s

    mean, sd, interval = print_mc("PROBLEM 1 (Cylindricity)", C_s, "um")
    validate_gum(C, U, interval, uc)
    plot_histogram(C_s, C, uc, interval, "Cylindricity C (μm)",
                   "Problem 1: PDF of cylindricity (Monte Carlo, M = 2×10^5)",
                   "problem1_cylindricity_histogram.png")
    return C, uc, U, mean, sd, interval


# ======================================================================
# PROBLEM 2 : ZINC CONCENTRATION   c = 1000 * m * P / V
# ======================================================================
def problem2():
    # Purity: Uniform [0.9985, 1.0005]
    P_min, P_max = 0.9985, 1.0005
    P = (P_min + P_max) / 2
    u_P = (P_max - P_min) / (2 * np.sqrt(3))

    # Mass: Gaussian
    m, u_m = 300.75, 0.009                                  # mg

    # Volume = V_fill + dV_rep + dV_temp
    V_min, V_max, V_mode = 149.6, 150.4, 150.0              # Triangular
    V0 = V_mode
    u_Vfill = (V_max - V_min) / 2 / np.sqrt(6)

    range_rep = 0.09                                        # Rectangular, full range 0.09 mL
    a_rep = range_rep / 2                                   # -> +/-0.045 mL
    u_Vrep = a_rep / np.sqrt(3)

    U_temp95 = 0.1                                          # Normal, 0.1 mL at 95 %
    u_Vtemp = U_temp95 / stats.norm.ppf(0.975)              # divide by 1.96

    V = V0
    u_V = np.sqrt(u_Vfill ** 2 + u_Vrep ** 2 + u_Vtemp ** 2)

    # ---- GUM ----
    cZn = 1000 * m * P / V
    c_m = 1000 * P / V                      # dc/dm
    c_P = 1000 * m / V                      # dc/dP
    c_V = -1000 * m * P / V ** 2            # dc/dV (same for each volume component)

```

```
    contrib = [c_P * u_P, c_m * u_m, c_V * u_Vfill, c_V * u_Vrep, c_V * u_Vtemp]
    uc = np.sqrt(sum(x ** 2 for x in contrib))
    nu_eff = np.inf                          # all inputs treated as known PDFs
    k = stats.norm.ppf(0.975)

    rows = [("Purity P", P, "B", "Uniform", u_P, c_P),
            ("Mass m (mg)", m, "B", "Gaussian", u_m, c_m),
            ("Volume - filling (mL)", V0, "B", "Triangular", u_Vfill, c_V),
            ("Volume - repeat. (mL)", 0.0, "A", "Rectangular", u_Vrep, c_V),
            ("Volume - temp. (mL)", 0.0, "B", "Normal", u_Vtemp, c_V)]
    U = print_budget("PROBLEM 2 (Zinc concentration)", rows, cZn, uc, nu_eff, k, "mg/L")
    print(f"(Combined u(V) = {u_V:.5f} mL, relative uc = {uc / cZn * 100:.4f} %)")

    # ---- Monte Carlo ----
    P_s = rng.uniform(P_min, P_max, M)
    m_s = rng.normal(m, u_m, M)
    V_s = (rng.triangular(V_min, V_mode, V_max, M)
           + rng.uniform(-a_rep, a_rep, M)
           + rng.normal(0.0, u_Vtemp, M))
    c_s = 1000 * m_s * P_s / V_s

    mean, sd, interval = print_mc("PROBLEM 2 (Zinc concentration)", c_s, "mg/L")
    validate_gum(cZn, U, interval, uc)
    plot_histogram(c_s, cZn, uc, interval, "Zinc concentration $c_{Zn}$ (mg/L)",
                   "Problem 2: PDF of zinc concentration (Monte Carlo, M = 2×10^5)",
                   "problem2_zinc_histogram.png")
    return cZn, uc, U, mean, sd, interval


# ======================================================================
def summary(name, unit, res):
    y, uc, U, mean, sd, (lo, hi) = res
    print(f"\n{name}")
    print(f"{'':<12}{'Estimate':>14}{'Std. unc.':>14}{'95 % interval':>32}")
    print(f"{'GUM':<12}{y:>14.4f}{uc:>14.4f}{'[' + f'{y - U:.4f}, {y + U:.4f}' + ']':>32}")
    print(f"{'Monte Carlo':<12}{mean:>14.4f}{sd:>14.4f}{'[' + f'{lo:.4f}, {hi:.4f}' + ']':>32}  ({unit})
    ↪ ")


if __name__ == "__main__":
    r1 = problem1()
    r2 = problem2()
    print("\n" + "=" * 92)
    print("SUMMARY: GUM vs MONTE CARLO")
    print("=" * 92)
    summary("Problem 1 - Cylindricity", "um", r1)
    summary("Problem 2 - Zinc concentration", "mg/L", r2)
    plt.show()
```